Operating Systems – Unit 3 Assignment

Banana Program (Memory Management)

Name: Alakh Gupta

SRN: PES2UG24CS051

Section: A

Date: 16 April 2026

SECTION 0 – System Information

Screenshot 0

```
Alakh@DESKTOP-J43B5JT:~$ export PS1="PES2UG24CS051$ "
PES2UG24CS051$ free -m
              total        used        free      shared  buff/cache   available
Mem:           7792         507        7287           3         147        7284
Swap:          2048           0        2048
PES2UG24CS051$
```

Q0.1 Answer:

Total memory = 7792 MB

= 7792 × 1024 × 1024

= 8178890752 bytes

Page size = 4096 bytes

Number of page frames = Total memory / page size

= 8178890752 / 4096

= 1997798 frames

SECTION 1 – Virtual Address Space

Screenshot 1

```
PES2UG24CS051$ nano layout.c
PES2UG24CS051$ gcc -o layout layout.c -no-pie
PES2UG24CS051$ ./layout
Text (main): 0x401216
Rodata: 0x402008
Data (init): 0x404050
BSS (uninit): 0x404064
Heap (malloc): 0x236622a0
Stack (Local): 0x7ffee7150364
Heap-to-Stack gap: 134212666 MB
--- /proc/self/maps (filtered) ---
00400000-00401000 r--p 00000000 /home/Alakh/Layout
00401000-00402000 r-xp 00001000 /home/Alakh/Layout
00402000-00403000 r--p 00002000 /home/Alakh/Layout
```

```
00403000-00404000 r--p 00002000 /home/Alakh/Layout
00404000-00405000 rw-p 00003000 /home/Alakh/Layout
23662000-23683000 rw-p 00000000 [heap]
7ffee7130000-7ffee7152000 rw-p 00000000 [stack]
PES2UG24CS051$
```

Q1.1 Answer:

The order of memory addresses is:

Text < Rodata < Data < BSS < Heap << Stack

Yes, this matches the expected virtual memory layout.

Q1.2 Answer:

The heap has permissions rw-p (read and write, private). It is not executable to prevent security vulnerabilities such as code injection attacks.

Q1.3 Answer:

Yes, the addresses change between runs due to ASLR. ASLR randomizes memory locations to improve system security and prevent attacks.

SECTION 2 – Demand Paging

Screenshot 2

```
PES2UG24CS051$ nano demand.c
PES2UG24CS051$ gcc -O0 -o demand demand.c
PES2UG24CS051$ ./demand
[1. before malloc]
VmSize:  2684 kB
VmRSS:   1536 kB
VmSwap:     0 kB
[2. after malloc, before touch]
VmSize: 264832 kB
VmRSS:   1536 kB
VmSwap:     0 kB
[3. after touching 64 MB]
VmSize: 264832 kB
VmRSS:  67072 kB
VmSwap:     0 kB
[4. after touching all 256 MB]
VmSize: 264832 kB
VmRSS: 263680 kB
VmSwap:     0 kB
PES2UG24CS051$
```

Q2.1 Answer:

After malloc, VmSize increases significantly but VmRSS increases very little. This is because memory is allocated virtually but not physically until accessed (demand paging).

Q2.2 Answer:

The number of page faults is approximately equal to the number of pages accessed. 256 MB / 4 KB ≈ 65536 page faults, with slight variation due to additional faults.

Q2.3 Answer:

The program does not use 256 MB immediately after malloc. VmSize represents allocated virtual memory, while RSS shows actual physical memory usage. Due to demand paging, physical memory is used only when pages are accessed.

SECTION 3 – Page Tables (pagemap)

Screenshot 3

```
PES2UG24CS051$ nano pagemap.c
PES2UG24CS051$ gcc -O0 -o pagemap pagemap.c
PES2UG24CS051$ sudo ./pagemap
[sudo] password for Alakh:
Before touching any page
page[0]: VA=0x60d601b7f2a0 present=1 swapped=0 PFN=0x145a7c
page[1]: VA=0x60d601b802a0 present=0 swapped=0 (no physical frame)
page[2]: VA=0x60d601b812a0 present=0 swapped=0 (no physical frame)
After touching pages 0 and 1
page[0]: VA=0x60d601b7f2a0 present=1 swapped=0 PFN=0x145a7c
page[1]: VA=0x60d601b802a0 present=1 swapped=0 PFN=0x137f83
page[2]: VA=0x60d601b812a0 present=0 swapped=0 (no physical frame)
PES2UG24CS051$
```

Q3.1 Answer:

Before touching, some pages have present = 0. After touching, page[0] and page[1] have present = 1, while page[2] remains 0. This shows that physical memory is allocated only when the page is accessed.

Q3.2 Answer:

Page 2 is not present and has no PFN. This shows that memory allocated by malloc is not mapped to physical memory until accessed.

Q3.3 Answer:

PFNs may differ between runs. This is because the operating system assigns available physical frames dynamically.

SECTION 4 – Copy-on-Write (COW)

Screenshot 4

```
PES2UG24CS051$ nano cow.c
PES2UG24CS051$ gcc -O0 -o cow cow.c
PES2UG24ACS051$ ./cow
Parent: allocated and touched 128 MB (32768 pages)
Child READ-ONLY scan:
  Page faults: 4
Child WRITE pass:
  Page faults: 32769
  Expected: 32768
PES2UG24CS051$
```

Q4.1 Answer:

The number of page faults during the read-only scan is very small (around 4). This is because both parent and child share the same memory pages without copying.

Q4.2 Answer:

The number of page faults during the write pass is approximately equal to the number of pages (around 32768). Each write operation triggers a copy-on-write fault, creating a separate copy of each page.

Q4.3 Answer:

Without copy-on-write, an additional 128 MB of physical memory would be required. COW is important because it avoids unnecessary copying when fork is followed by exec.

SECTION 5 – Shared Libraries

Screenshot 5

```
PES2UG24CS051$ ldd /bin/bash
        linux-vdso.so.1 (0x00007fffb42e2000)
        libtinfo.so.6 => /lib/x86_64-linux-gnu/libtinfo.so.6 (0x0000714152ad1000)
        libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x0000714152800000)
        /lib64/ld-linux-x86-64.so.2 (0x0000714152c78000)
PES2UG24CS051$ nano hello.c
PES2UG24CS051$ gcc -o hello_dynamic hello.c
PES2UG24CS051$ gcc -static -o hello_static hello.c
PES2UG24CS051$ ls -lh hello_dynamic hello_static
-rwxr-xr-x 1 Alakh Alakh  16K Apr  2 09:03 hello_dynamic
-rwxr-xr-x 1 Alakh Alakh 767K Apr  2 09:03 hello_static
PES2UG24CS051$ LD_DEBUG=libs ./hello_dynamic 2>&1 | head -30
      1524:     find library=libc.so.6 [0]; searching
      1524:     search cache=/etc/ld.so.cache
      1524:     trying file=/lib/x86_64-linux-gnu/libc.so.6
      1524:
      1524:     calling init: /lib64/ld-linux-x86-64.so.2
      1524:
      1524:     calling init: /lib/x86_64-linux-gnu/libc.so.6
      1524:
      1524:     initialize program: ./hello_dynamic
      1524:
      1524:     transferring control: ./hello_dynamic
      1524:
      1524:     calling fini: [0]
      1524:
      1524:     calling fini: /lib/x86_64-linux-gnu/libc.so.6
PES2UG24CS051$
```

Q5.1 Answer:

The static binary is much larger than the dynamic binary. This is because static linking includes all library code inside the executable, while dynamic linking uses shared libraries.

Q5.2 Answer:

The different permissions correspond to:

r--p: read-only data

r-xp: executable code

rw-p: writable data

These represent different segments of the shared library.

Q5.3 Answer:

The OS keeps only one copy of the shared library code in memory for all processes. However, each process has its own copy of writable data.

SECTION 6 – How malloc Gets Memory

Screenshot 6 (Part 1)

```
PES2UG24CS051$ nano alloc_trace.c
PES2UG24CS051$ gcc -O0 -o alloc_trace alloc_trace.c
PES2UG24CS051$ ./alloc_trace
Initial break: 0x5c5ec2101000
After malloc(1KB):   break=0x5c5ec2122000 ptr=0x5c5ec21016b0
After malloc(256KB): break=0x5c5ec2122000 ptr=0x7e94af713010
After malloc(512B):  break=0x5c5ec2122000 ptr=0x5c5ec2101ac0
small  at 0x5c5ec21016b0
small2 at 0x5c5ec2101ac0
large  at 0x7e94af713010
```

Screenshot 6 (Part 2)

```
PES2UG24CS051$ strace -e brk,mmap ./alloc_trace 2>&1 | tail -20
mmap(NULL, 19823, PROT_READ, MAP_PRIVATE, 3, 0) = 0x76fe3e277000
mmap(NULL, 2170256, PROT_READ, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0x76fe3e000000
mmap(0x76fe3e028000, 1605632, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
0x28000) = 0x76fe3e028000
mmap(0x76fe3e1b0000, 323584, PROT_READ, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1b0000) =
0x76fe3e1b0000
mmap(0x76fe3e1ff000, 24576, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3,
0x1fe000) = 0x76fe3e1ff000
mmap(0x76fe3e205000, 52624, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1,
0) = 0x76fe3e205000
mmap(NULL, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x76fe3e274000
brk(NULL) = 0x58526c63b000
brk(0x58526c65c000) = 0x58526c65c000
mmap(NULL, 266240, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x76fe3e233000
Initial break: 0x58526c63b000
After malloc(1KB):   break=0x58526c65c000 ptr=0x58526c63c2b0
PES2UG24CS051$
```

Q6.1 Answer:

The program break does not change after malloc(256 KB). This is because large allocations are handled using mmap instead of brk.

Q6.2 Answer:

Small allocations use brk, while large allocations use mmap. The threshold is approximately 128 KB in glibc.

Q6.3 Answer:

Using only brk would lead to fragmentation, as freed memory cannot be reused efficiently. mmap allows large allocations to be managed independently, reducing fragmentation.

SECTION 7 – Fragmentation

Screenshot 7

```
PES2UG24CS051$ nano fragment.c
PES2UG24CS051$ gcc -O0 -o fragment fragment.c
PES2UG24CS051$ ./fragment
Pool: 1024 blocks of 4096 bytes = 4096 KB total
Freed every other block: 512 free blocks (2048 KB free)
Looking for 128 contiguous free blocks (512 KB)...
FAILED - 2048 KB free total, but largest contiguous run < 512 KB
This is EXTERNAL FRAGMENTATION.
--- Internal Fragmentation ---
malloc(1) at 0x6159977fb6b0, malloc(1) at 0x6159977fb6d0
Distance between them: 32 bytes
PES2UG24CS051$
```

Q7.1 Answer:

Total free memory is 2048 KB, but it is not contiguous. The 512 KB request fails because free memory is split into small non-contiguous blocks.

Q7.2 Answer:

The checkerboard pattern prevents coalescing because no two free blocks are adjacent. Hence, larger contiguous memory blocks cannot be formed.

Q7.3 Answer:

Paging eliminates external fragmentation because physical memory does not need to be contiguous. The page table maps virtual pages to any available physical frames.

SECTION 8 – Swapping

Screenshot 8

```
PES2UG24CS051$ swapon --show
NAME      TYPE      SIZE USED PRIO
/dev/sde  partition   2G   0B   -2
PES2UG24CS051$ free -m
              total        used        free      shared  buff/cache   available
Mem:           7792         540        6998           3         410        7251
Swap:          2048           0        2048
PES2UG24CS051$
```

Q8.1 Answer:

Swap activity occurs when memory usage increases. The system moves pages between RAM and disk to manage memory efficiently.

Q8.2 Answer:

VmSwap represents memory that has been moved to disk. The kernel swaps out less frequently used pages to free physical memory.

Q8.3 Answer:

Major page faults are slow because they require disk access. Minor faults are faster as they involve only memory operations.

SECTION 9 – Page Replacement

Screenshot 9

```
PES2UG24CS051$ grep -E 'Active|Inactive' /proc/meminfo
Active:           183468 kB
Inactive:         293132 kB
Active(anon):       2676 kB
Inactive(anon):    92524 kB
Active(file):     180792 kB
Inactive(file):   200608 kB
PES2UG24CS051$
```

Q9.1 Answer:

The smaller working set is faster because it fits in memory. The larger working set requires more memory access and reduces performance.

Q9.2 Answer:

Active pages are frequently used and kept in memory. Inactive pages are less frequently used and may be replaced.

Q9.3 Answer:

Linux does not use true LRU because it is expensive to track every memory access. Instead, it uses an approximation based on accessed bits for efficiency.

SECTION 10 – Thrashing

Screenshot 10

```
PES2UG24CS051$ free -m
              total        used        free      shared  buff/cache   available
Mem:           7792         540        6997         411        7251
Swap:          2048           0        2048
PES2UG24CS051$
```

Due to system limitations, the full thrashing program was not executed, but the concept is explained.

Q10.1 Answer:

Throughput decreases significantly when memory usage exceeds available RAM. Performance drops sharply due to excessive paging.

Q10.2 Answer:

During thrashing, swap activity (si/so) increases and CPU waits for I/O operations. This leads to high I/O wait time and poor performance.

Q10.3 Answer:

The OOM killer terminates processes when the system runs out of memory. Swappiness controls how aggressively the kernel swaps memory between RAM and disk.